

NX Cred

Tài liệu đặc tả ứng dụng

Ứng dụng định danh sử dụng bằng chứng không tiết lộ
theo tiêu chuẩn AnonCreds

Phiên bản demo • September 16, 2026

Abstract

Đây là bộ tài liệu đặc tả về **NX Cred** — ứng dụng định danh sử dụng **bảng chứng không tiết lộ**, áp dụng tiêu chuẩn của **AnonCreds**.

Trong mô hình định danh hiện nay, mỗi lần một người chứng minh danh tính là một lần dữ liệu cá nhân của họ bị sao chép thêm vào một cơ sở dữ liệu mới. Người dùng mất quyền kiểm soát, bên tiếp nhận gánh rủi ro lưu trữ, và bên cấp giấy tờ vô tình trở thành nơi nắm giữ toàn bộ lịch sử hoạt động của công dân.

NX Cred thay việc đưa dữ liệu bằng việc đưa một bảng chứng toán học về dữ liệu. Hệ thống gồm ba bên — **Issuer**, **Holder** và **Verifier** — cùng một **blockchain** giữ khoá công khai. Bên xác minh khẳng định được tấm thẻ là thật, do đúng cơ quan có thẩm quyền ký, và đúng là của người đang đứng trước mặt, mà không nhìn thấy bất kỳ trường dữ liệu nào ngoài những trường người dùng chủ động mở.

Tài liệu mô tả đầy đủ: cấu trúc hệ thống và các khái niệm nền tảng, bài toán và lý do chọn giải pháp, bốn luồng hoạt động giữa các bên, toàn bộ công thức của lỗi mật mã, thiết kế bảo mật, thiết kế blockchain, ba tình huống ứng dụng thực tế, cùng những giới hạn hiện tại và hướng phát triển.

Cách đọc tài liệu này. Các thuật ngữ được đánh dấu bằng liên kết dẫn về chương **1** (*Topology*) — nơi thuật ngữ đó được định nghĩa. Bạn bấm vào từ chưa rõ để nhảy tới định nghĩa, rồi quay lại đọc tiếp. Chương **5** chứa toàn bộ phần toán học và có thể bỏ qua nếu không cần đi sâu.

Mục lục

Abstract	i
1 Topology	1
1.1 Cấu trúc hệ thống	1
1.2 Các bên tham gia	1
1.2.1 Issuer — bên cấp	1
1.2.2 Holder — ví người dùng	1
1.2.3 Verifier — bên xác minh	2
1.2.4 Blockchain — sổ cái công khai	2
1.3 Khái niệm nền tảng	2
1.3.1 Self-sovereign identity — định danh tự chủ	2
1.3.2 AnonCreds	2
1.3.3 Zero-knowledge proof — bằng chứng không tiết lộ	2
1.3.4 Bhutan NDI	2
1.4 Đại lượng mật mã	2
1.4.1 Big integer — số nguyên lớn	2
1.4.2 Link secret	3
1.4.3 Blinding factor — hệ số làm mù	3
1.4.4 Commitment — cam kết	3
1.4.5 Nonce — số dùng một lần	3
1.4.6 Thách thức (challenge) và Fiat–Shamir	3
1.4.7 Sigma protocol — giao thức sigma	3
1.4.8 CL signature — chữ ký Camenisch–Lysyanskaya	3
1.4.9 Blind signing — ký mù	3
1.4.10 Unblinding — gỡ mù	3
1.4.11 Credential — thẻ định danh	4
1.4.12 Schema — khuôn mẫu giấy tờ	4
1.4.13 Khoá công khai — credential definition	4
1.4.14 Modulus	4
1.4.15 Safe prime	4
1.4.16 Randomization — ngẫu nhiên hoá chữ ký	4
1.5 Tính chất và cơ chế	4
1.5.1 Selective disclosure — tiết lộ chọn lọc	4
1.5.2 Unlinkability — tính không liên kết được	4
1.5.3 Presentation request — yêu cầu trình bày	4
1.5.4 Predicate proof — bằng chứng vị từ	5
1.5.5 eKYC	5
1.5.6 Passkey	5
1.5.7 Revocation — thu hồi	5
2 Introduction	6
2.1 AnonCreds là tiêu chuẩn gì	6
2.2 Zero-knowledge proof là thuật toán gì	6

3	Giới thiệu bài toán	8
3.1	User story	8
3.2	NX Cred là giải pháp, và vì sao	8
4	Mô tả luồng hoạt động	10
4.1	Tổng quan luồng hoạt động	10
4.2	Luồng chi tiết: Issuer và Wallet	11
4.3	Luồng chi tiết: Wallet và Verifier	12
4.4	Luồng chi tiết: Verifier và Issuer	12
4.5	Luồng chi tiết: Issuer, Blockchain và Verifier	13
5	Lỗi mật mã ứng dụng	14
5.1	Tham số và thiết lập khoá	14
5.2	Công thức và tính toán giữa Issuer và Wallet	14
5.2.1	Cam kết	14
5.2.2	Giao thức sigma	14
5.2.3	Ký mù và gỡ mù	15
5.3	Công thức và tính toán giữa Wallet và Verifier	15
5.3.1	Ngẫu nhiên hoá và làm mù	15
5.3.2	Phép kiểm của Verifier	16
5.4	Công thức và tính toán giữa Issuer và Verifier	16
6	Thiết kế bảo mật	17
6.1	Mỗi tài khoản một link secret duy nhất	17
6.2	Dữ liệu người dùng chỉ nằm trên máy của họ	17
6.3	Không để lộ thông tin qua thông báo lỗi	17
6.4	Bảo vệ bằng passkey ngay trên thiết bị	17
6.5	Thiết kế ba bên công khai	17
7	Thiết kế blockchain	19
7.1	Issuer chỉ công bố đúng hai thứ	19
7.1.1	Khoá công khai gồm những gì	19
7.1.2	Schema là gì	19
7.2	Verifier dùng schema để soạn yêu cầu	19
7.3	Vì sao phải là blockchain	20
8	Các use case	21
8.1	Bhutan NDI — đã triển khai thực tế	21
8.2	Chống thi hộ tại các trường đại học Việt Nam	21
8.2.1	Cách NX Cred giải quyết	21
8.2.2	Vì sao cơ chế này chặn được thi hộ	21
8.3	Ngân hàng và dịch vụ tài chính	22
9	Kết luận	23
9.1	Giới hạn hiện tại	23
10	Hướng phát triển tương lai	24
	Tham khảo	25
A	Các thư viện sử dụng trong ứng dụng	26
A.1	Phía máy chủ — Python	26
A.2	Phía trình duyệt — TypeScript	26

B	Source code	28
C	Video demo và hướng dẫn sử dụng	29

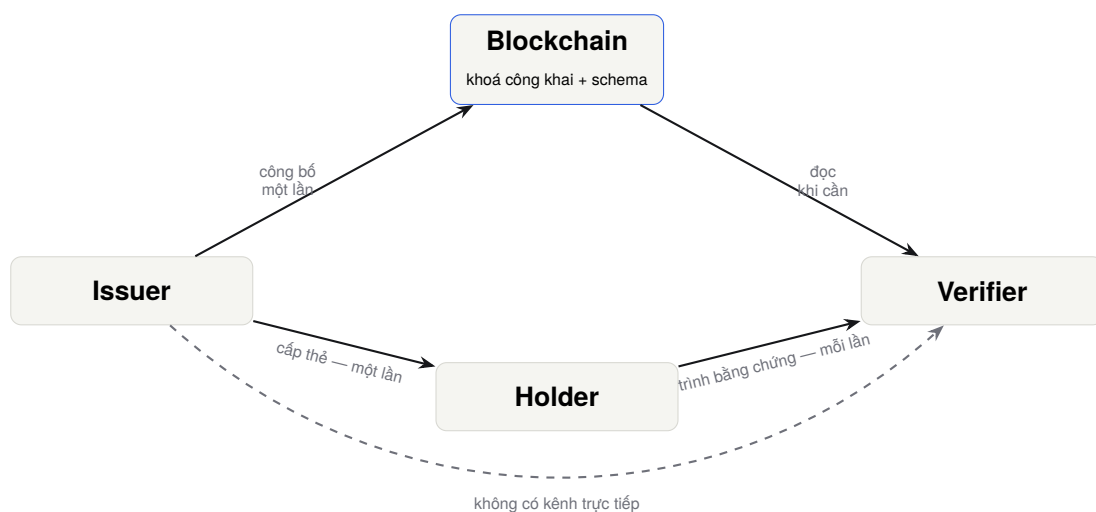
Chương 1

Topology

Chương này trình bày cấu trúc hệ thống và định nghĩa các khái niệm dùng trong tài liệu. Đây cũng là nơi các liên kết thuật ngữ ở các chương sau dẫn về.

1.1 Cấu trúc hệ thống

NX Cred gồm bốn thành phần: ba bên tham gia và một sổ cái công khai.



Hai đặc điểm của cấu trúc này quyết định mọi thứ phía sau. Thứ nhất, Issuer và Verifier không có kênh liên lạc trực tiếp: bên xác minh không hỏi lại cơ quan cấp mà tự đọc khoá công khai trên blockchain. Thứ hai, việc trình bằng chứng diễn ra thẳng giữa ví và bên xác minh, không đi qua blockchain, nên sổ cái không giữ lịch sử ai xác minh cái gì.

1.2 Các bên tham gia

1.2.1 Issuer — bên cấp

Bên có thẩm quyền cao nhất trong việc xác nhận danh tính: cơ quan quản lý căn cước, trường đại học, hoặc tổ chức phát hành giấy tờ. Issuer đối chiếu hồ sơ **eKYC**, ký **credential** bằng **chữ ký mù**, và công bố **khóa công khai** cùng **schema** lên **blockchain**.

1.2.2 Holder — ví người dùng

Phần mềm phía người dùng, còn gọi là *wallet*. Holder sinh và giữ **link secret**, nhận và lưu **credential**, rồi tạo **bằng chứng không tiết lộ** mỗi khi cần chứng minh danh tính. Đây là bên duy nhất có thể tạo ra bằng chứng hợp lệ.

1.2.3 Verifier — bên xác minh

Bất kỳ tổ chức nào cần kiểm tra danh tính: ngân hàng, khách sạn, nhà tuyển dụng, hội đồng thi. Verifier soạn [yêu cầu trình bày](#) dựa trên [schema](#) lấy từ blockchain, rồi kiểm bằng chứng nhận được bằng [khóa công khai](#) cũng lấy từ blockchain.

1.2.4 Blockchain — sổ cái công khai

Nơi [issuer](#) công bố [khóa công khai](#) và [schema](#). Điểm mấu chốt là dữ liệu trên đó *không ai sửa được*, kể cả chính issuer. Blockchain trong NX Cred **không** lưu dữ liệu cá nhân, **không** lưu credential, và **không** ghi lại các lần xác minh.

1.3 Khái niệm nền tảng

1.3.1 Self-sovereign identity — định danh tự chủ

Mô hình định danh trong đó người dùng — chứ không phải một tổ chức nào — là bên nắm giữ và kiểm soát dữ liệu định danh của chính mình. Người dùng quyết định chia sẻ gì, với ai, trong bao lâu. NX Cred được xây theo mô hình này.

1.3.2 AnonCreds

Viết tắt của *Anonymous Credentials*: bộ tiêu chuẩn mô tả cách cấp và trình bày giấy tờ số sao cho bên xác minh kiểm chứng được tính hợp lệ mà không đọc được dữ liệu, và không thể ghép hai lần trình bày lại với nhau. Tiêu chuẩn này quy định bốn thành phần chính: [link secret](#), [chữ ký CL](#), [tiết lộ chọn lọc](#), và [tính không liên kết được](#). AnonCreds đã được vương quốc Bhutan áp dụng cho hệ thống định danh quốc gia [Bhutan NDI](#) phục vụ khoảng 200.000 người dân.

1.3.3 Zero-knowledge proof — bằng chứng không tiết lộ

Thuật toán mật mã cho phép chứng minh một mệnh đề là đúng mà không tiết lộ bất cứ điều gì ngoài chính tính đúng đắn đó. Một bằng chứng như vậy phải thoả ba tính chất: *đầy đủ* (điều đúng thì luôn chứng minh được), *chắc chắn* (điều sai thì không thể chứng minh, kể cả khi kẻ gian cố gắng tính toán lớn), và *không tiết lộ* (bên kiểm kết thúc phiên mà không biết thêm gì). Đây là trái tim của NX Cred.

1.3.4 Bhutan NDI

Hệ thống định danh số quốc gia của vương quốc Bhutan, xây trên tiêu chuẩn [AnonCreds](#). Đây là minh chứng thực tế rằng mô hình này vận hành được ở quy mô quốc gia chứ không chỉ trong phòng thí nghiệm.

1.4 Đại lượng mật mã

1.4.1 Big integer — số nguyên lớn

Số nguyên có độ dài hàng trăm đến hàng nghìn chữ số nhị phân — lớn hơn rất nhiều so với kiểu số thông thường của máy tính (thường 64 bit). Toàn bộ phép toán trong NX Cred làm việc trên những số như vậy: [modulus](#) dài khoảng 2048 bit, tức xấp xỉ 617 chữ số thập phân. Độ lớn này chính là thứ khiến việc phá giải bằng cách thử mọi khả năng trở nên bất khả thi.

1.4.2 Link secret

Một **số nguyên lớn** ngẫu nhiên dài 256 bit, được sinh ra ngay sau khi người dùng tạo ví, và *không bao giờ rời khỏi thiết bị* — kể cả **issuer** cũng không nhìn thấy. **Credential** được ký sao cho chỉ dùng được cùng với con số này. Hệ quả: kẻ nào chép trộm được file credential mà không có link secret thì cũng không tạo được bằng chứng hợp lệ. Mỗi tài khoản tương ứng với đúng một link secret duy nhất.

1.4.3 Blinding factor — hệ số làm mù

Một **số nguyên lớn** ngẫu nhiên do ví sinh ra để che giấu giá trị thật trong một **cam kết**. Vì mỗi giá trị bí mật giả định đều tương thích với đúng một blinding factor, cam kết gửi đi không mang thông tin nào về giá trị thật. Ví giữ blinding factor lại để về sau **gỡ mù** chữ ký.

1.4.4 Commitment — cam kết

Một số được tính từ giá trị bí mật cộng với **blinding factor**, đóng vai như chiếc phong bì dán kín: người khác ký được lên phong bì mà không đọc được nội dung bên trong, và người gửi về sau không thể tráo đổi nội dung đó.

1.4.5 Nonce — số dùng một lần

Số ngẫu nhiên do bên kiểm phát ra, chỉ dùng đúng một lần rồi huỷ. Nonce tham gia vào phép tính **thách thức**, nên một bằng chứng cũ gắn với nonce cũ không thể được phát lại cho phiên mới. Đây là cơ chế chống tấn công phát lại (replay attack).

1.4.6 Thách thức (challenge) và Fiat–Shamir

Trong một giao thức chứng minh, bên kiểm gửi một câu hỏi ngẫu nhiên và bên chứng minh phải trả lời. Biến đổi *Fiat–Shamir* thay câu hỏi đó bằng một giá trị băm của chính dữ liệu bên chứng minh vừa tạo ra. Vì không đoán trước được giá trị băm, bên chứng minh không thể dựng sẵn câu trả lời giả — nhờ vậy giao thức từ hai chiều trở thành một chiều, gửi một lần là xong.

1.4.7 Sigma protocol — giao thức sigma

Dạng giao thức ba bước — cam kết, thách thức, đáp số — dùng để chứng minh *biết* một bí mật mà không nói ra bí mật đó. NX Cred dùng sigma protocol ở bước xin cấp credential, để ví chứng minh mình thật sự nắm **link secret** và **blinding factor** nằm bên trong **cam kết** vừa gửi.

1.4.8 CL signature — chữ ký Camenisch–Lysyanskaya

Lược đồ chữ ký số cho phép ký lên nhiều thuộc tính cùng lúc, ký lên cả phần bị che, và cho phép người giữ chữ ký về sau **ngẫu nhiên hoá** lại nó mà vẫn giữ tính hợp lệ. Chính hai tính chất cuối khiến CL signature phù hợp cho định danh ẩn danh, điều mà chữ ký RSA hay ECDSA thông thường không làm được.

1.4.9 Blind signing — ký mù

Cách ký trong đó bên ký không nhìn thấy một phần nội dung mình đang ký, nhưng chữ ký vẫn ràng buộc chặt vào phần bị che đó. Trong NX Cred, **issuer** ký lên **cam kết** chứa **link secret** mà không hề biết link secret là gì.

1.4.10 Unblinding — gỡ mù

Thao tác ví thực hiện sau khi nhận chữ ký từ issuer: dùng **blinding factor** đã giữ lại để biến chữ ký trên cam kết thành chữ ký hoàn chỉnh trên dữ liệu thật.

1.4.11 Credential — thẻ định danh

Kết quả cuối cùng của quá trình cấp: bộ ba số (A, e, v) tạo thành chữ ký của issuer lên toàn bộ thuộc tính cộng với [link secret](#). Đây là thứ ví lưu lại và dùng để tạo bằng chứng về sau.

1.4.12 Schema — khuôn mẫu giấy tờ

Danh sách các trường mà một loại giấy tờ có. Ví dụ với thẻ sinh viên: mã số sinh viên, họ và tên, mã lớp, ngày hết hạn. Schema được [issuer](#) công bố lên [blockchain](#), và nó là ranh giới cứng: issuer không ký gì ngoài các trường trong schema, [verifier](#) cũng không hỏi được gì ngoài các trường đó.

1.4.13 Khoá công khai — credential definition

Bộ số issuer công bố để người khác kiểm chữ ký của mình. Trong NX Cred gồm: [modulus](#) n , ba cơ số S, R, Z , và một cơ số riêng R_i cho mỗi trường trong [schema](#). Chi tiết xem mục [5.1](#).

1.4.14 Modulus

Số $n = p \cdot q$ với p, q là hai [safe prime](#) bí mật. Mọi phép nhân và lũy thừa trong hệ thống đều thực hiện modulo n . Biết n nhưng không biết p, q thì không ký được — đó là nền tảng an toàn của lược đồ.

1.4.15 Safe prime

Số nguyên tố dạng $p = 2p' + 1$ trong đó p' cũng là số nguyên tố. Dùng safe prime khiến nhóm tính toán có bậc chứa thừa số nguyên tố lớn, chặn được các thuật toán tấn công khai thác nhóm có bậc phân tích được thành nhiều thừa số nhỏ.

1.4.16 Randomization — ngẫu nhiên hoá chữ ký

Thao tác ví thực hiện trước mỗi lần trình bày: biến chữ ký A thành $A' = A \cdot S^r$ với r ngẫu nhiên mới. Chữ ký gửi đi vì thế mỗi lần một khác, tạo ra [tính không liên kết được](#).

1.5 Tính chất và cơ chế

1.5.1 Selective disclosure — tiết lộ chọn lọc

Khả năng mở đúng những trường được hỏi trên một tấm thẻ và giữ kín phần còn lại, trong khi bên xác minh vẫn kiểm chứng được tấm thẻ là trọn vẹn và hợp lệ. Các trường không mở vẫn tham gia vào phép tính dưới dạng đã che.

1.5.2 Unlinkability — tính không liên kết được

Tính chất khiến hai lần trình bày cùng một credential không thể bị ghép lại với nhau. Đạt được nhờ [ngẫu nhiên hoá](#) chữ ký trước mỗi lần gửi: không có mã số cố định nào đi theo người dùng, nên hai verifier gộp dữ liệu cũng không tìm ra điểm chung.

1.5.3 Presentation request — yêu cầu trình bày

Bản yêu cầu verifier gửi cho ví, gồm: những trường cần được tiết lộ, những mệnh đề cần được chứng minh mà không tiết lộ, và một [nonce](#).

1.5.4 Predicate proof — bằng chứng vị từ

Dạng bằng chứng chứng minh một quan hệ so sánh trên thuộc tính số mà không lộ giá trị — ví dụ “năm sinh ≤ 2007 ” để chứng minh đủ 18 tuổi. NX Cred hiện *chưa* hiện thực phần này; xem mục [9.1](#).

1.5.5 eKYC

Viết tắt của *electronic Know Your Customer*: bước đối chiếu danh tính điện tử, trong đó người dùng quét giấy tờ và bên cấp so với hồ sơ mình đang giữ. Trong NX Cred, đây là bước duy nhất dữ liệu định danh được đối chiếu trực tiếp, và nó chỉ diễn ra *một lần* trong đời một tài khoản.

1.5.6 Passkey

Khoá sinh trắc học theo chuẩn WebAuthn (vân tay, khuôn mặt, mã PIN thiết bị) dùng để mở kho lưu trữ của ví. Cần phân biệt rõ: passkey là lớp bảo vệ *quyền truy cập* vào thiết bị, không phải cơ chế mật mã tạo nên bằng chứng.

1.5.7 Revocation — thu hồi

Cơ chế cho phép issuer vô hiệu hoá một credential đã cấp. NX Cred hiện *chưa* có phần này; xem mục [9.1](#).

Chương 2

Introduction

2.1 AnonCreds là tiêu chuẩn gì

[AnonCreds](#) là bộ tiêu chuẩn mô tả cách cấp phát và trình bày giấy tờ số sao cho ba yêu cầu sau đồng thời được thỏa mãn:

1. **Bên xác minh tin được.** Bằng chứng nhận về phải chứng tỏ giấy tờ do đúng cơ quan có thẩm quyền ký, chưa bị sửa, và thuộc về đúng người đang trình.
2. **Người dùng không lộ thừa.** Chỉ những trường được hỏi và được người dùng đồng ý mới hiện ra; phần còn lại vẫn nằm trong phép tính nhưng ở dạng đã che.
3. **Không ai theo dõi được.** Hai lần trình cùng một tấm thẻ phải không ghép lại được với nhau, và bên cấp giấy tờ không được biết thẻ đang dùng ở đâu.

Ba yêu cầu này xung đột nhau nếu dùng chữ ký số thông thường. Chữ ký RSA hay ECDSA là một giá trị cố định: đưa ra lần nào cũng như lần nào, nên nó chính là một mã số theo dõi hoàn hảo. Muốn giấu bớt thuộc tính thì chữ ký không còn kiểm được. AnonCreds giải quyết bằng bốn thành phần:

[Link secret](#)

gắn credential với đúng một ví, khiến bản sao bị đánh cắp trở nên vô dụng.

[Ký mù](#)

cho phép issuer ký lên link secret mà không nhìn thấy nó.

[Tiết lộ chọn lọc](#)

cho phép mở từng trường riêng lẻ.

[Ngẫu nhiên hoá](#)

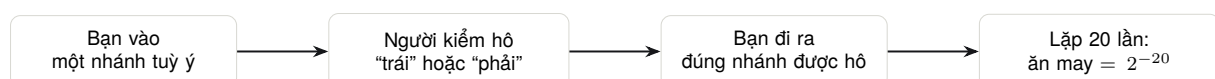
khiến mỗi lần trình bày là một lần hoàn toàn mới.

Tiêu chuẩn này không chỉ nằm trên giấy. Vương quốc Bhutan đã triển khai [Bhutan NDI](#) trên nền AnonCreds, phục vụ khoảng 200.000 người dân.

2.2 Zero-knowledge proof là thuật toán gì

[Zero-knowledge proof](#) là thuật toán cho phép chứng minh một mệnh đề đúng mà không tiết lộ vì sao nó đúng.

Để hình dung, hãy xét một ví dụ không cần tới toán. Có một hang động hình vòng, lối vào chia làm hai nhánh trái và phải, hai nhánh nối nhau ở cuối bằng một cánh cửa khoá. Bạn tuyên bố mình biết mật khẩu cánh cửa đó. Người kiểm đứng ngoài, không thấy bạn chọn nhánh nào. Bạn đi vào một nhánh tùy ý; sau đó người kiểm hô “trái” hoặc “phải”, và bạn phải đi ra bằng nhánh được hô.



Nếu bạn thật sự biết mật khẩu, bạn luôn ra đúng nhánh. Nếu không biết, bạn chỉ ra đúng khi may mắn đã chọn sẵn nhánh đó — xác suất 50%. Lặp lại 20 lần, xác suất ăn may là 2^{-20} , tức khoảng một phần triệu. Người kiểm buộc phải tin bạn biết mật khẩu, dù chưa từng nghe mật khẩu.

Ví dụ này chứa đủ ba tính chất của một bằng chứng không tiết lộ:

- **Đầy đủ** — người biết mật khẩu luôn qua được.
- **Chắc chắn** — người không biết gần như chắc chắn bị phát hiện.
- **Không tiết lộ** — người kiểm không học được gì về mật khẩu.

Điểm mấu chốt là bên chứng minh phải phản ứng với một **thách thức ngẫu nhiên** mà họ không đoán trước được. Trong hệ thống thật, thách thức đó là một giá trị băm, và việc lặp lại 20 lần được thay bằng việc dùng một con số đủ lớn — 256 bit — để xác suất ăn may nhỏ tới mức không đáng kể.

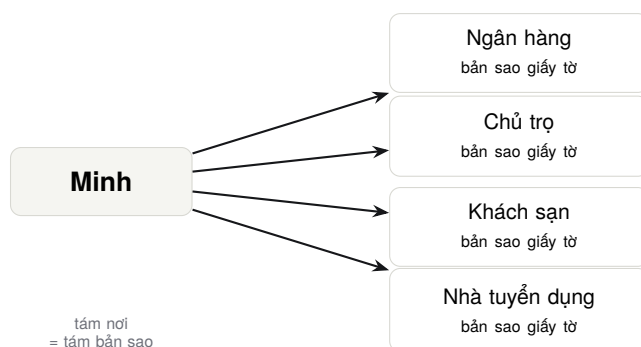
Chương 3

Giới thiệu bài toán

3.1 User story

Minh là sinh viên năm cuối. Trong sáu tháng, Minh phải chứng minh danh tính ở tám nơi khác nhau: mở tài khoản ngân hàng để nhận lương thực tập, ký hợp đồng thuê trọ, làm thẻ thư viện liên trường, đăng ký thi chứng chỉ ngoại ngữ, nhận phòng khách sạn khi đi thực tập xa, mua sim điện thoại, đăng ký dịch vụ ví điện tử, và nộp hồ sơ xin việc.

Mỗi lần như vậy, Minh đưa căn cước ra. Bên kia chụp lại hoặc photo. Sau sáu tháng, dữ liệu định danh đầy đủ của Minh — số căn cước, ngày sinh, quê quán, địa chỉ thường trú — nằm trong tám cơ sở dữ liệu mà Minh không kiểm soát, không biết được bảo vệ ra sao, và không thể yêu cầu xóa.



Ba vấn đề lộ ra:

Nhân bản không thu hồi được

Tám bản sao nghĩa là tám cơ hội rò rỉ. Chỉ cần một nơi bị tấn công là dữ liệu của Minh ra ngoài, và không có cách nào “thu hồi” một tấm căn cước đã bị chụp. Tệ hơn, Minh thường không biết vụ rò rỉ đã xảy ra.

Lộ thừa thông tin

Khách sạn chỉ cần biết Minh là người thật và đủ tuổi thuê phòng. Nhưng tấm căn cước nói cho họ luôn cả quê quán và địa chỉ nhà. Minh trả lời một câu hỏi bằng cách tiết lộ mười câu trả lời — và không có lựa chọn nào khác.

Giám sát phát sinh ngoài ý muốn

Ở những hệ thống có kiểm tra trực tuyến, mỗi lần Minh dùng giấy tờ là một lần bên tiếp nhận hỏi lại cơ quan cấp. Lặp lại đủ nhiều, cơ quan cấp trở thành nơi nắm được bản đồ hoạt động của Minh: khi nào mở tài khoản, thuê nhà ở đâu, đi những đâu. Không ai thiết kế hệ thống với mục đích giám sát, nhưng kết quả vẫn là giám sát.

3.2 NX Cred là giải pháp, và vì sao

NX Cred đảo ngược mô hình: thay vì đưa dữ liệu rồi mong bên kia giữ gìn, người dùng đưa một **bằng chứng toán học** về dữ liệu đó.

Vấn đề	Cơ chế	Kết quả
Nhân bản dữ liệu	Bằng chứng thay cho bản sao	Bên tiếp nhận không có gì để làm rõ rĩ
Lộ thừa thông tin	Tiết lộ chọn lọc	Chỉ mở đúng trường được hỏi
Giám sát phát sinh	Khoá công khai trên blockchain	Issuer không cần và không thể biết
Thẻ bị đánh cắp	Link secret trên máy	Bản sao credential vô dụng
Bị ghép dấu vết	Ngẫu nhiên hoá mỗi lần	Hai lần trình không liên kết được

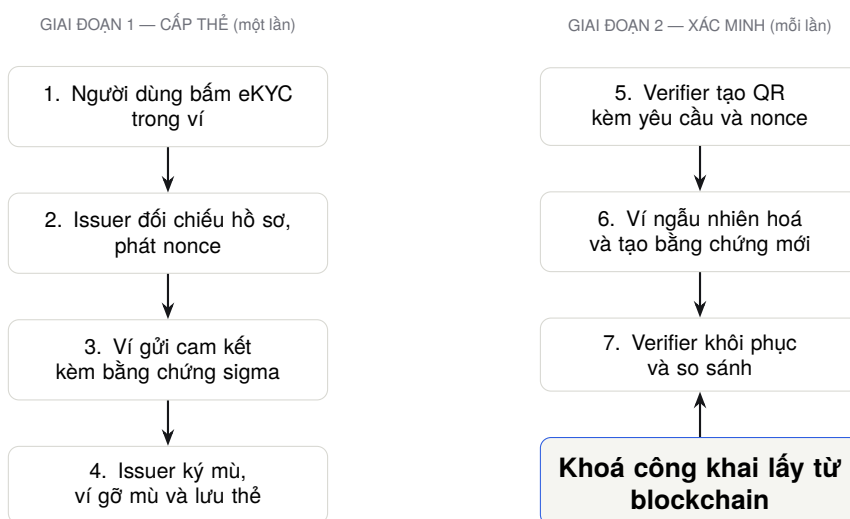
Quay lại câu chuyện của Minh: sau khi dùng NX Cred, Minh làm [eKYC](#) đúng *một lần* với cơ quan cấp. Tám nơi còn lại nhận về tám bằng chứng toán học khác nhau. Không nơi nào giữ dữ liệu gốc, không nơi nào ghép được với nơi khác, và cơ quan cấp không biết Minh đã đi những đâu.

Chương 4

Mô tả luồng hoạt động

4.1 Tổng quan luồng hoạt động

Toàn hệ thống vận hành qua bảy bước, chia làm hai giai đoạn tách biệt: *cấp thẻ* diễn ra một lần, *xác minh* diễn ra mỗi khi cần.

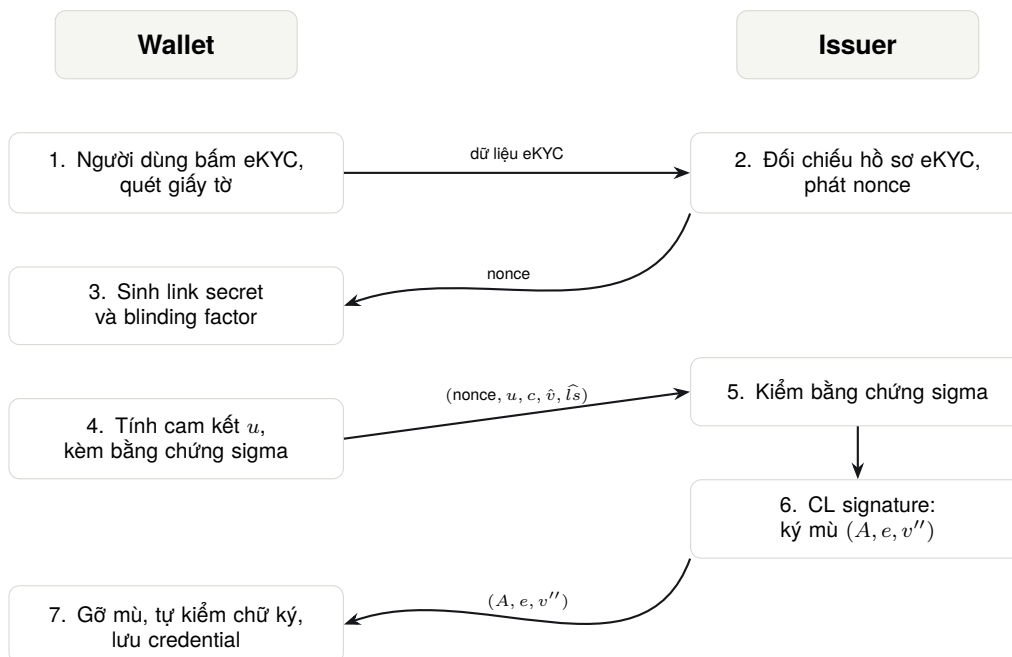


Diễn giải từng bước. Người dùng bấm **eKYC** trong ví để yêu cầu được cấp thẻ; ví gửi dữ liệu giấy tờ cho **Issuer** đối chiếu. Issuer đối chiếu hồ sơ và phát về một **nonce**, đồng thời yêu cầu ví tạo một **bằng chứng** chứng minh nó thật sự sở hữu **link secret** và **blinding factor** nằm trong **cam kết** sắp gửi. Ví gửi lại nonce kèm bằng chứng đó; Issuer kiểm, nếu hợp lệ thì thực hiện **CL signature**, tức **ký mù** lên cam kết và toàn bộ thuộc tính. Ví nhận chữ ký về, **gỡ mù**, tự kiểm lại, rồi lưu thành **credential**. Giai đoạn cấp thẻ kết thúc và không lặp lại.

Khi cần xác minh, **Verifier** tạo mã QR chứa **yêu cầu trình bày** kèm một nonce mới. Ví **ngẫu nhiên hoá** chữ ký, làm mù các thuộc tính không chia sẻ, rồi tạo bằng chứng — mỗi lần yêu cầu là một lần làm mù lại và tạo lại từ đầu. Verifier nhận bằng chứng, khôi phục lại giá trị cần so sánh và đối chiếu, dùng **khoá công khai** đọc từ **blockchain**.

4.2 Luồng chi tiết: Issuer và Wallet

Đây là giai đoạn cấp thẻ, chạy đúng một lần cho mỗi người dùng.



Bước 4 dễ bị bỏ qua nhưng không thể thiếu.

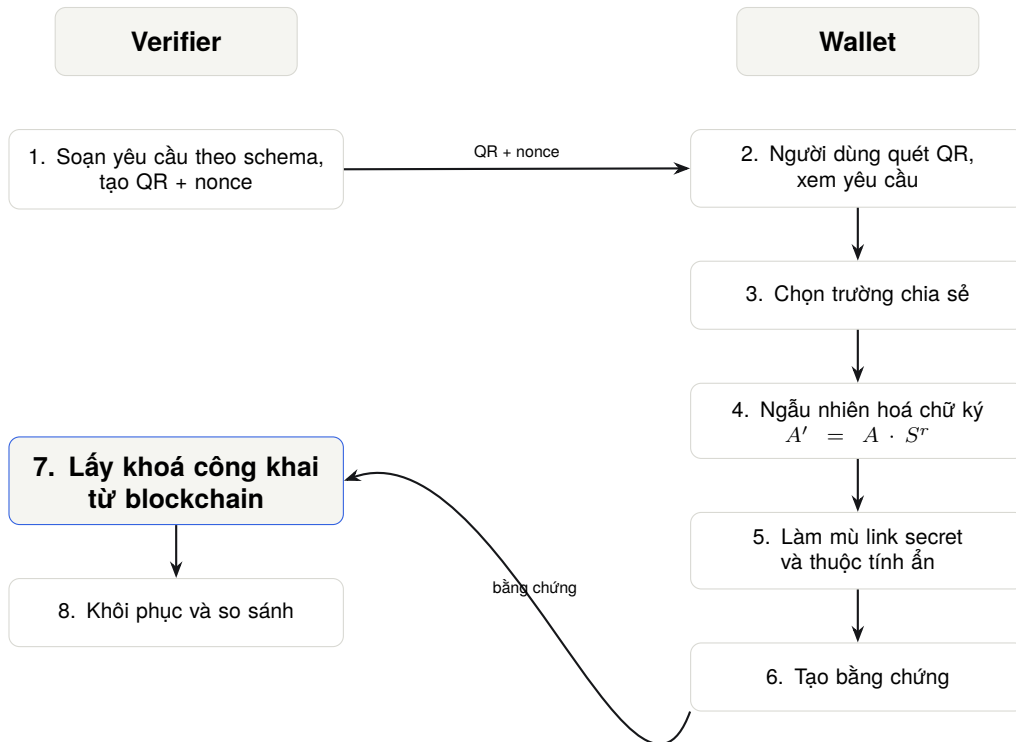
Nếu ví chỉ gửi **cam kết** u mà không kèm bằng chứng, một kẻ gian có thể chặn bắt cam kết của người khác rồi nộp như của mình. Hắn sẽ nhận về một credential gắn với **link secret** mà hắn không biết — vô dụng với hắn, nhưng gây rối loạn sổ sách và tiêu mất suất cấp thẻ của nạn nhân. **Sigma protocol** ở bước này buộc người nộp phải thật sự biết cặp số nằm bên trong cam kết.

Bước 7 cũng vậy.

Ví kiểm lại chữ ký trước khi lưu. Nếu issuer trả về một chữ ký hỏng — do lỗi hoặc cố ý — ví phát hiện ngay tại chỗ, thay vì đến lúc đi xác minh mới hỏng và không biết quy trách nhiệm cho ai.

4.3 Luồng chi tiết: Wallet và Verifier

Đây là luồng chính, chạy xuyên suốt vòng đời ứng dụng.

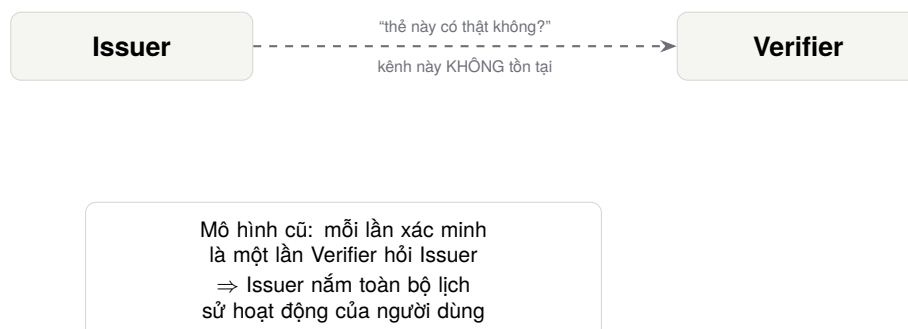


Điểm quan trọng nhất của luồng này: các bước 4–6 chạy lại *từ đầu* cho mỗi yêu cầu. Ví không lưu sẵn một bằng chứng rồi dùng lại. Nhờ vậy hai verifier khác nhau nhận về hai bằng chứng không có điểm chung nào, và một bằng chứng bị bắt trộm cũng không dùng lại được cho **nonce** khác.

Về mặt sản phẩm, bước 3 đáng nhấn mạnh: quyền quyết định thuộc về người dùng. Verifier chỉ được *hỏi*; ví hỏi lại chủ của nó trước khi trả lời. Những trường không nằm trong yêu cầu thì bị khoá cứng trên giao diện — người dùng không thể vô tình chia sẻ.

4.4 Luồng chi tiết: Verifier và Issuer

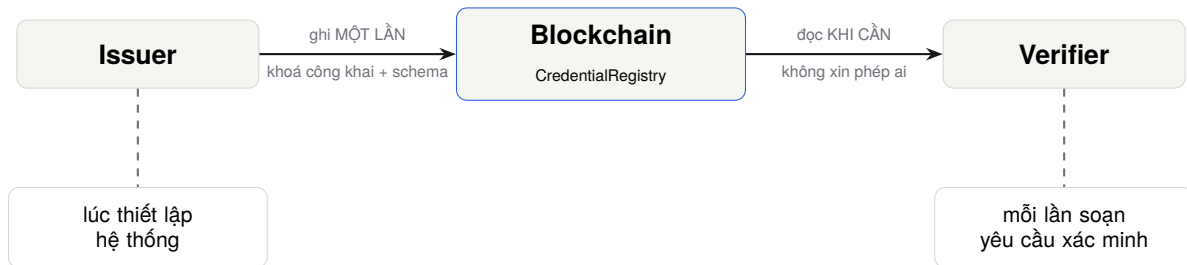
Nội dung của mục này là một sự vắng mặt: **Verifier** và **Issuer** không có kênh liên lạc trực tiếp nào.



Trong mô hình truyền thống, Verifier phải gọi về Issuer để hỏi giấy tờ này có thật không. Câu hỏi đó tạo ra ba hệ quả xấu: Issuer biết được toàn bộ hoạt động của người dùng; hệ thống sập nếu Issuer ngừng hoạt động; và mỗi lần xác minh phải chờ một vòng mạng.

NX Cred cắt đứt kênh đó bằng cách để Issuer công bố **khóa công khai** lên **blockchain** một lần duy nhất. Verifier tự đọc, tự kiểm, không cần xin phép và không cần Issuer còn trực tuyến. Issuer không biết, và về mặt kỹ thuật là không thể biết, thẻ đang được dùng ở đâu.

4.5 Luồng chi tiết: Issuer, Blockchain và Verifier



Quy trình gồm hai thao tác, cách nhau về thời gian có thể là hàng tháng:

1. **Issuer ghi lên chain.** Khi thiết lập hệ thống, issuer sinh cặp khóa rồi ghi phần công khai cùng **schema** vào hợp đồng. Thao tác này thực hiện một lần.
2. **Verifier đọc từ chain.** Khi cần xác minh, verifier đọc schema để biết loại giấy tờ này có những trường nào — từ đó soạn được yêu cầu hợp lệ — và đọc khóa công khai để kiểm bằng chứng.

Điểm cốt lõi về mặt an ninh: verifier **không bao giờ** nhận khóa công khai từ bên đang trình bằng chứng. Nếu nhận, kẻ gian chỉ việc tự sinh cặp khóa của mình, tự ký một tấm thẻ giả, rồi nộp kèm khóa công khai của chính hắn — mọi phép kiểm sẽ đều qua. Lấy khóa từ một nguồn độc lập và không sửa được là thứ duy nhất chặn đường tấn công này.

Chương 5

Lỗi mật mã ứng dụng

Chương này có thể bỏ qua hoàn toàn nếu bạn không muốn tìm hiểu sâu. Toàn bộ phần còn lại của tài liệu đọc được mà không cần tới đây.

Mọi phép nhân và lũy thừa dưới đây đều hiểu là trong $\mathbb{Z}_n^*$, tức modulo n , trừ khi ghi rõ là phép toán trên $\mathbb{Z}$.

5.1 Tham số và thiết lập khoá

Issuer chọn hai **safe prime** p, q dài 1024 bit, đặt $n = pq$ và $\varphi = (p-1)(q-1)$. Lấy ngẫu nhiên $a, b, z, r_i \in [0, n)$ rồi đặt

$$S = a^2, \quad R = b^2, \quad Z = z^2, \quad R_i = r_i^2 \pmod{n}.$$

Bình phương mọi cơ sở đưa chúng vào nhóm các thặng dư bình phương, bảo đảm mọi phần tử tham gia phép kiểm nằm cùng một nhóm con. Bộ $(n, S, R, Z, \{R_i\})$ chính là **khóa công khai** được ghi lên **blockchain**; cặp (p, q) giữ kín tuyệt đối.

Thuộc tính được mã hoá thành số bằng hàm

$$\text{enc}(x) = \begin{cases} \text{int}(x) & \text{nếu } x \text{ là chuỗi toàn chữ số,} \\ \text{int}(\text{SHA-256}(x)) & \text{ngược lại,} \end{cases}$$

và **thách thức** tính bằng $H(x, y, \text{nonce}) = \text{int}(\text{SHA-256}(x||y||\text{nonce}))$.

5.2 Công thức và tính toán giữa Issuer và Wallet

5.2.1 Cam kết

Ví sinh **link secret** ls dài 256 bit và **blinding factor** v' dài 2048 bit, rồi tính

$$u = S^{v'} R^{ls} \pmod{n}. \quad (5.1)$$

Định lý 5.1 (Cam kết giấu kín hoàn hảo). *Với mỗi giá trị giả định $\widehat{ls}$ đều tồn tại đúng một $\widehat{v}$ sao cho $S^{\widehat{v}} R^{\widehat{ls}} = u$. Do đó u không mang thông tin nào về ls .*

Chứng minh. Gọi k là bậc của S và giả sử $R = S^\alpha$. Khi đó $u = S^{v' + \alpha ls}$. Với $\widehat{ls}$ bất kỳ, chọn $\widehat{v} \equiv v' + \alpha ls - \alpha \widehat{ls} \pmod{k}$ thì $S^{\widehat{v}} R^{\widehat{ls}} = S^{v' + \alpha ls} = u$. Nghiệm duy nhất modulo k , nên phân phối của u độc lập với ls . $\square$

5.2.2 Giao thức sigma

Ví lấy $\tilde{v}$ dài 3488 bit, $\tilde{ls}$ dài 593 bit rồi tính

$$\tilde{u} = S^{\tilde{v}} R^{\tilde{ls}}, \quad c = H(u, \tilde{u}, \text{nonce}), \quad \hat{v} = \tilde{v} + c v', \quad \hat{ls} = \tilde{ls} + c ls,$$

với hai phép cộng cuối thực hiện trên $\mathbb{Z}$. Ví gửi $(\text{nonce}, u, c, \hat{v}, \hat{ls})$ và không gửi $\tilde{u}$.

Issuer kiểm $1 < u < n$, $\gcd(u, n) = 1$, rồi tính $\tilde{u}' = S^{\hat{v}} R^{\hat{ls}} u^{-c}$ và chấp nhận khi $H(u, \tilde{u}', \text{nonce}) = c$.

Định lý 5.2 (Tính đầy đủ). Với ví trung thực thì $\tilde{u}' = \tilde{u}$.

Chứng minh.

$$S^{\hat{v}} R^{\hat{ls}} u^{-c} = S^{\tilde{v}+cv'} R^{\tilde{ls}+cls} (S^{v'} R^{ls})^{-c} = S^{\tilde{v}} R^{\tilde{ls}} = \tilde{u}.$$

Mọi thành phần nhân với c triệt tiêu. $\square$

Định lý 5.3 (Tính chắc chắn). Ai trả lời đúng cho hai thách thức khác nhau $c_1 \neq c_2$ trên cùng một $\tilde{u}$ thì thực sự biết ls .

Chứng minh. Chia hai đẳng thức kiểm cho nhau được $S^{\hat{v}_1-\hat{v}_2} R^{\hat{ls}_1-\hat{ls}_2} = u^{c_1-c_2}$. Đặt $\Delta c = c_1 - c_2 \neq 0$, ta rút ra $ls = (\hat{ls}_1 - \hat{ls}_2)/\Delta c$ và $v' = (\hat{v}_1 - \hat{v}_2)/\Delta c$ — một mở cam kết hợp lệ của u . $\square$

5.2.3 Ký mù và gỡ mù

Issuer chọn số nguyên tố $e \in [2^{596}, 2^{596} + 2^{119})$ với $\gcd(e, \varphi) = 1$, lấy v'' dài 2724 bit, tính $d = e^{-1} \bmod \varphi$ rồi đặt

$$Q = Z \cdot \left(u S^{v''} \prod_i R_i^{m_i} \right)^{-1}, \quad A = Q^d \bmod n. \quad (5.2)$$

Ví nhận (A, e, v'') , đặt $v = v' + v''$ và lưu **credential** (A, e, v) .

Định lý 5.4 (Đẳng thức chữ ký CL).

$$Z = A^e S^v R^{ls} \prod_i R_i^{m_i} \pmod{n}. \quad (5.3)$$

Chứng minh. Vì $de \equiv 1 \pmod{\varphi}$ nên $A^e = Q^{de} = Q$. Thay Q và $u = S^{v'} R^{ls}$:

$$A^e S^v R^{ls} \prod_i R_i^{m_i} = Z \left(S^{v'} R^{ls} S^{v''} \prod_i R_i^{m_i} \right)^{-1} S^{v'+v''} R^{ls} \prod_i R_i^{m_i} = Z. \quad \square$$

Đây là bất biến trung tâm của toàn hệ thống: sai một thuộc tính, sai link secret, hay chữ ký giả — đều không cho ra Z .

5.3 Công thức và tính toán giữa Wallet và Verifier

5.3.1 Ngẫu nhiên hoá và làm mù

Ví lấy r dài 2128 bit rồi đặt

$$A' = A S^r, \quad v^* = v - er, \quad e^* = e - 2^{596}. \quad (5.4)$$

Bổ đề 5.5 (Đẳng thức được bảo toàn). $Z = A'^e S^{v^*} R^{ls} \prod_i R_i^{m_i}$.

Chứng minh. Từ $A = A' S^{-r}$ suy ra $A^e = A'^e S^{-er}$. Thay vào Định lý 5.4, phần S^{-er} gộp vào số mũ của S biến v thành $v - er = v^*$. $\square$

Gọi $\mathcal{H}$ là tập thuộc tính giữ kín, $\mathcal{R}$ là tập tiết lộ. Ví lấy $\tilde{e}$ (456 bit), $\tilde{v}$ (3060 bit), $\tilde{m}_{ls}$ và $\tilde{m}_i$ (593 bit mỗi số) rồi tính

$$T = A'^{\tilde{e}} S^{\tilde{v}} R^{\tilde{m}_{ls}} \prod_{i \in \mathcal{H}} R_i^{\tilde{m}_i}, \quad c = H(T, A', n_v), \quad (5.5)$$

$$\hat{e} = \tilde{e} + ce^*, \quad \hat{v} = \tilde{v} + cv^*, \quad \hat{m}_{ls} = \tilde{m}_{ls} + cl_s, \quad \hat{m}_i = \tilde{m}_i + cm_i \quad (i \in \mathcal{H}).$$

Mọi số che đều dài hơn số bị che ít nhất 80 bit. Vì $\tilde{s}$ phân bố đều trên một khoảng độ dài $2^{\ell_{\tilde{s}}}$ còn cs chỉ là một hằng số trong khoảng $2^{\ell_s + \ell_c}$, khoảng cách thống kê giữa $\tilde{s} + cs$ và số ngẫu nhiên thuần túy nhỏ hơn 2^{-80} .

5.3.2 Phép kiểm của Verifier

Verifier kiểm biên ($1 < A' < n$, $0 \leq \hat{e} < 2^{457}$, $0 \leq c < 2^{256}$, tập thuộc tính khớp yêu cầu, enc của giá trị thô khớp m_i), rồi tính

$$D = Z \left(\prod_{i \in \mathcal{R}} R_i^{m_i} \right)^{-1} A'^{-2^{596}}, \quad \hat{T} = D^{-c} A'^{\hat{e}} S^{\hat{v}} R^{\hat{m}_{ls}} \prod_{i \in \mathcal{H}} R_i^{\hat{m}_i}, \quad (5.6)$$

và chấp nhận khi $H(\hat{T}, A', n_v) = c$.

Bổ đề 5.6. $D = A'^{e^*} S^{v^*} R^{ls} \prod_{i \in \mathcal{H}} R_i^{m_i}$.

Chứng minh. Từ Bổ đề 5.5 và $e = e^* + 2^{596}$, chuyển hai thừa số mà verifier tự tính được sang về trái là ra đúng biểu thức của D . $\square$

Định lý 5.7 (Tính đúng đắn). Với bằng chứng trung thực, $\hat{T} = T$.

Chứng minh.

$$\begin{aligned} A'^{\hat{e}} S^{\hat{v}} R^{\hat{m}_{ls}} \prod_{i \in \mathcal{H}} R_i^{\hat{m}_i} &= A'^{\hat{e} + ce^*} S^{\hat{v} + cv^*} R^{\hat{m}_{ls} + cls} \prod_{i \in \mathcal{H}} R_i^{\hat{m}_i + cm_i} \\ &= \underbrace{\left(A'^{\hat{e}} S^{\hat{v}} R^{\hat{m}_{ls}} \prod_{i \in \mathcal{H}} R_i^{\hat{m}_i} \right)}_T \cdot \underbrace{\left(A'^{e^*} S^{v^*} R^{ls} \prod_{i \in \mathcal{H}} R_i^{m_i} \right)^c}_{D^c \text{ (Bổ đề 5.6)}} = T D^c. \end{aligned}$$

Nhân hai vế với D^{-c} được $\hat{T} = T$, nên hàm băm cho ra đúng c . $\square$

5.4 Công thức và tính toán giữa Issuer và Verifier

Giữa hai bên này không có phép tính chung nào, vì không có kênh liên lạc nào. Toàn bộ quan hệ toán học giữa họ nằm ở chỗ: verifier sử dụng đúng bộ số $(n, S, R, Z, \{R_i\})$ mà issuer đã công bố, và đẳng thức chữ ký ở Định lý 5.4 chỉ đúng khi bằng chứng thực sự bắt nguồn từ một credential do issuer đó ký.

Nói cách khác, blockchain đóng vai kênh truyền *một chiều*, *bất đồng bộ*, *không cần tin tưởng* giữa issuer và toàn bộ verifier trên thế giới.

Chương 6

Thiết kế bảo mật

6.1 Mỗi tài khoản một link secret duy nhất

Ngay sau khi người dùng tạo ví, hệ thống sinh đúng một **link secret** và một **blinding factor** cho tài khoản đó. Hai con số này là **số nguyên lớn** ngẫu nhiên, không dẫn xuất từ mật khẩu hay bất kỳ dữ liệu cá nhân nào, nên không thể đoán được.

Hệ quả thiết kế: **credential** không phải là một tấm vé vô danh dùng chung. Nó gắn chặt với đúng một link secret. Một bản sao credential rơi vào tay người khác là một mớ số vô nghĩa, vì họ không tạo nổi bằng chứng nếu thiếu link secret tương ứng.

6.2 Dữ liệu người dùng chỉ nằm trên máy của họ

Sau bước **eKYC** một lần duy nhất, dữ liệu định danh gốc không được gửi đi đâu nữa. Mọi lần xác minh về sau chỉ trao đổi bằng chứng toán học.

Điều này khác biệt về bản chất so với mô hình “gửi ảnh giấy tờ”: ở đây không tồn tại bản sao nào để mà rò rỉ. Bên xác minh có bị tấn công thì kẻ tấn công cũng chỉ thu được những bằng chứng đã dùng, vốn đã hết hạn vì gắn với **nonce** cũ, và không chứa dữ liệu gốc.

6.3 Không để lộ thông tin qua thông báo lỗi

Khi dữ liệu người dùng khai không khớp hồ sơ trong cơ sở dữ liệu, hệ thống chỉ báo một câu ngắn rằng thông tin không khớp và mời quét lại. Thông báo tuyệt đối không nêu trường nào bị lệch.

Lý do là một dạng tấn công gọi là *oracle*: nếu hệ thống trả lời “sai nơi thường trú” thì kẻ dò biết ngay các trường còn lại đã đúng, và chỉ cần thử lần lượt từng trường là dựng lại được toàn bộ hồ sơ của nạn nhân mà không cần xâm nhập cơ sở dữ liệu. Một thông báo hữu ích cho người dùng trung thực có thể là một công cụ dò tìm cho kẻ tấn công, nên ở đây phải chọn phía an toàn.

6.4 Bảo vệ bằng passkey ngay trên thiết bị

Kho lưu trữ của ví được khoá bằng **passkey** theo chuẩn WebAuthn — vân tay, khuôn mặt, hoặc mã PIN của chính thiết bị. Khoá riêng của passkey nằm trong phần cứng bảo mật của máy và không trích xuất ra được.

Cần nói rõ ranh giới: passkey bảo vệ *quyền truy cập* vào ví. Nó không tham gia vào phép toán tạo bằng chứng. Hai lớp này độc lập nhau, và việc nhầm lẫn giữa chúng là hiểu sai phổ biến nhất về hệ thống.

Vì bằng chứng gắn với thiết bị đang giữ link secret, người dùng **không cần** quay video sinh trắc hay chụp ảnh khuôn mặt mỗi lần xác minh. Một bằng chứng là đủ để khẳng định “tôi là tôi”.

6.5 Thiết kế ba bên công khai

Vai trò của ba bên được định nghĩa công khai và ràng buộc bằng mật mã, không bằng thoả thuận:

Bên	Làm được gì	Không làm được gì
Issuer	Ký credential cho người đã qua eKYC	Không thấy link secret; không biết thẻ dùng ở đâu; không giả được thẻ cho người chưa eKYC
Holder	Tạo bằng chứng, chọn trường chia sẻ	Không sửa được thuộc tính đã ký; không tạo được bằng chứng cho credential mình không giữ
Verifier	Kiểm bằng chứng, biết chắc thẻ hợp lệ	Không thấy trường không được mở; không ghép được hai lần trình bày

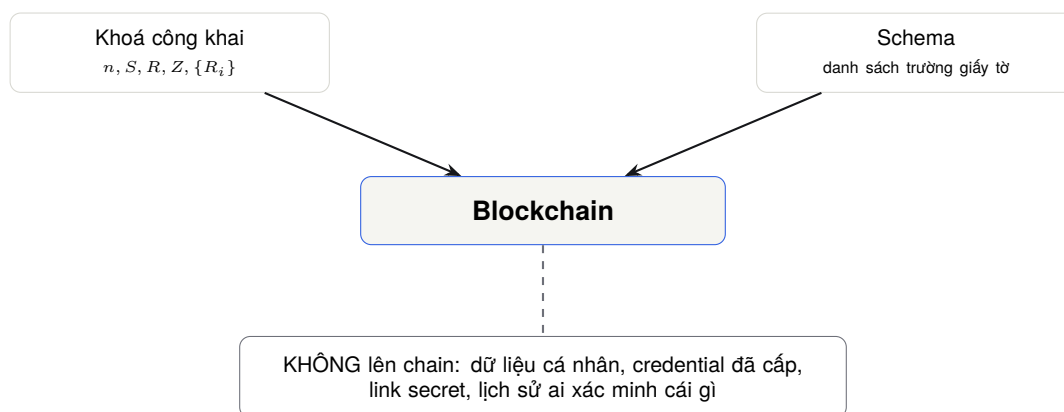
Không bên nào phải tin bên nào. Uy tín giữa các bên đến từ việc mọi ràng buộc đều kiểm chứng được độc lập, bằng dữ liệu công khai trên [blockchain](#).

Chương 7

Thiết kế blockchain

7.1 Issuer chỉ công bố đúng hai thứ

Issuer ghi lên chain duy nhất **khóa công khai** và **schema**. Không gì khác.



7.1.1 Khóa công khai gồm những gì

Chi tiết đầy đủ ở mục 5.1. Nói cho dễ hiểu: đó là một bộ số cho phép bất kỳ ai *kiểm* được chữ ký của issuer, nhưng không cho phép ai *tạo* ra chữ ký đó — giống như con dấu mà ai cũng nhận ra được nhưng không ai khắc lại được.

7.1.2 Schema là gì

Schema là khung mô tả một loại giấy tờ có những trường nào. Ví dụ với thẻ sinh viên:

Trường	Ý nghĩa
mssv	Mã số sinh viên
name	Họ và tên
class	Mã lớp
expiry	Ngày hết hạn

Trong bản demo hiện tại, schema được dùng là mặt trước thẻ căn cước công dân với tám trường: số thẻ, họ và tên, ngày sinh, giới tính, quốc tịch, quê quán, nơi thường trú, ngày hết hạn.

7.2 Verifier dùng schema để soạn yêu cầu

Vì schema nằm công khai trên chain, verifier biết chính xác loại giấy tờ này có gì, từ đó soạn được **yêu cầu trình bày** hợp lệ. Với thẻ sinh viên ở trên, verifier có thể hỏi:

- Người này có mã số sinh viên do trường cấp không?
- Thẻ sinh viên còn hạn không?

- Mã lớp có đúng là lớp đang thi không?

Verifier chỉ cần lấy schema từ chain về là đủ để làm việc — không phải liên hệ, không phải ký hợp đồng, không phải chờ ai cấp quyền.

7.3 Vì sao phải là blockchain

Lý do chỉ có một: **khoá công khai** phải nằm ở nơi không ai sửa được, kể cả chính **Issuer**. Nếu khoá đặt trên một máy chủ thường, người kiểm soát máy chủ đó có thể âm thầm thay khoá rồi ký thẻ giả mà không ai phát hiện. Blockchain ở đây đóng vai một bảng tin công khai không thể tẩy xoá, không phải một cơ sở dữ liệu.

Chương 8

Các use case

8.1 Bhutan NDI — đã triển khai thực tế

Bhutan NDI là hệ thống định danh số quốc gia của vương quốc Bhutan, xây trên tiêu chuẩn **AnonCreds** và đã phục vụ khoảng 200.000 người dân.

Giá trị mang lại:

- **Rút ngắn thời gian thủ tục.** Quy trình mở tài khoản ngân hàng vốn mất nhiều ngày chờ đối chiếu hồ sơ nay rút xuống còn vài phút, vì bằng chứng kiểm được tại chỗ.
- **Giảm gánh nặng lưu trữ cho doanh nghiệp.** Ngân hàng và nhà cung cấp dịch vụ không còn phải lưu bản sao giấy tờ, kéo theo giảm rủi ro pháp lý và chi phí tuân thủ.
- **Người dân giữ quyền kiểm soát.** Mỗi lần chia sẻ là một lần người dân chủ động đồng ý, với đúng những trường họ chọn.

8.2 Chống thi hộ tại các trường đại học Việt Nam

Tình trạng gian lận danh tính trong thi cử — thi hộ, mượn thẻ sinh viên — diễn ra khá phổ biến. Các biện pháp hiện nay đều có điểm yếu: kiểm thẻ giấy thì dễ làm giả, đối chiếu ảnh thì tốn thời gian và sai sót, còn thu thập sinh trắc học thì lại tạo ra một kho dữ liệu nhạy cảm mới.

8.2.1 Cách NX Cred giải quyết

Trước kỳ thi, nhà trường (đóng vai **issuer**) cấp cho mỗi sinh viên một **credential** chứa mã số sinh viên, mã lớp, ngành và ngày hết hạn. Tại phòng thi, giám thị (đóng vai **verifier**) tạo một **yêu cầu** với vài mệnh đề đơn giản:

- Có tên trong danh sách dự thi của môn này không?
- Có đúng mã lớp không?
- Thẻ sinh viên còn hiệu lực không?

Sinh viên quét mã QR, ví tạo bằng chứng, giám thị nhận kết quả trong vài giây.



8.2.2 Vì sao cơ chế này chặn được thi hộ

Điểm then chốt nằm ở **link secret**. Muốn nhờ người khác thi hộ, sinh viên phải giao cả ví lẫn link secret cho người đó. Nhưng link secret không phải một mã số dùng riêng cho kỳ thi — nó là thứ mở khoá *toàn bộ* credential trong ví: thẻ ngân hàng, giấy tờ tùy thân, mọi giấy tờ số khác.

Nói cách khác, cái giá của việc nhờ thi hộ là trao toàn bộ danh tính số của mình cho người khác. Chi phí đó đủ lớn để cơ chế tự bảo vệ chính nó, mà không cần tới camera hay sinh trắc học.

8.3 Ngân hàng và dịch vụ tài chính

Quy trình định danh khách hàng của ngân hàng hiện nay tốn kém ở cả hai phía: khách hàng phải nộp hồ sơ nhiều lần cho nhiều ngân hàng, còn ngân hàng phải lưu trữ và bảo vệ kho dữ liệu đó suốt vòng đời quan hệ khách hàng.

Với NX Cred, khách hàng làm eKYC một lần với cơ quan có thẩm quyền. Sau đó mỗi ngân hàng chỉ cần một **bằng chứng** rằng khách hàng có giấy tờ hợp lệ, do đúng cơ quan nhà nước cấp, và chưa hết hạn. Ngân hàng đạt được mục đích tuân thủ mà không phải trở thành nơi lưu trữ dữ liệu nhạy cảm — giảm đồng thời chi phí vận hành và rủi ro pháp lý khi có sự cố.

Chương 9

Kết luận

NX Cred cho thấy mô hình định danh không tiết lộ là khả thi trên thực tế, không chỉ trên lý thuyết. Hệ thống đạt được đồng thời ba điều mà mô hình truyền thống buộc phải đánh đổi: bên xác minh tin được kết quả, người dùng không lộ dữ liệu thừa, và không bên nào theo dõi được người dùng.

Cụ thể, bản hiện thực này đã chạy được: ký mù bằng chữ ký CL thật; sigma protocol khi xin cấp thẻ; tiết lộ chọn lọc theo từng trường; ngẫu nhiên hoá chữ ký tạo tính không liên kết được; và công bố khoá công khai lên blockchain để verifier kiểm độc lập.

9.1 Giới hạn hiện tại

Ứng dụng hiện tại mới chỉ chứng minh được một mệnh đề rất đơn giản: credential này đã được Issuer ký. Muốn áp dụng các loại bằng chứng khác thì phải thêm thuật toán tính toán. Cụ thể, bằng chứng vị từ kiểu “đủ 18 tuổi mà không lộ ngày sinh” đòi hỏi range proof, và “thuộc một trong các lớp sau” đòi hỏi set membership proof. Riêng với chứng minh tuổi, còn phải đổi cách mã hoá ngày sinh sang dạng số có thứ tự, vì hiện tại ngày sinh bị băm nên mất hoàn toàn quan hệ so sánh.

Ngoài ra Issuer cũng chưa có chức năng thu hồi, khiến cho nếu mất thiết bị là mất hết credential và phải xin cấp lại từ đầu. Nghiêm trọng hơn, một credential cấp nhầm hoặc cấp cho người sau đó mất tư cách vẫn tiếp tục có hiệu lực.

Một giới hạn thứ ba thuộc về kiến trúc: trong bản demo, phần mật mã của ví chạy trên máy chủ để rút ngắn thời gian dựng sản phẩm. Đúng tinh thần định danh tự chủ thì phần này phải chạy trong trình duyệt, để link secret không bao giờ rời khỏi thiết bị người dùng.

Chương 10

Hướng phát triển tương lai

1. **Giao diện soạn bằng chứng cho người không chuyên.** Mục tiêu là verifier chỉ cần bấm chọn từ danh sách mệnh đề dựng sẵn — “đủ 18 tuổi”, “còn hiệu lực”, “thuộc lớp X” — và hệ thống tự sinh yêu cầu tương ứng, thay vì phải hiểu cấu trúc [schema](#) và tự soạn.
2. **Bổ sung cơ chế thu hồi.** Thêm sổ đăng ký thu hồi (revocation registry) trên chain, để issuer vô hiệu hoá được credential và ví chứng minh được “credential của tôi chưa bị thu hồi” mà vẫn không lộ danh tính.
3. **Bằng chứng vị từ.** Hiện thực range proof cho các thuộc tính số, mở đường cho nhóm mệnh đề so sánh vốn là nhu cầu phổ biến nhất trong thực tế.
4. **Chuyển lõi mật mã về trình duyệt.** Biên dịch phần tính toán sang WebAssembly để chạy hoàn toàn phía người dùng, hoàn thiện mô hình tự chủ danh tính.

Tham khảo

1. AnonCreds Specification — <https://hyperledger.github.io/anoncreds-spec/>
2. J. Camenisch, A. Lysyanskaya. *A Signature Scheme with Efficient Protocols*, 2002.
3. W3C. *Verifiable Credentials Data Model* — <https://www.w3.org/TR/vc-data-model-2.0/>
4. W3C. *Web Authentication (WebAuthn) Level 3* — <https://www.w3.org/TR/webauthn-3/>
5. Bhutan National Digital Identity — <https://www.bhutanndi.com/>
6. Hyperledger Indy — <https://hyperledger-indy.readthedocs.io/>
7. A. Fiat, A. Shamir. *How to Prove Yourself: Practical Solutions to Identification and Signature Problems*, 1986.

Phụ lục A

Các thư viện sử dụng trong ứng dụng

A.1 Phía máy chủ — Python

Thư viện	Vai trò	Địa chỉ
gmpy2	Số học số nguyên lớn : sinh safe prime , lũy thừa modulo. Gánh toàn bộ phần nặng của lỗi mật mã.	https://pypi.org/project/gmpy2/
FastAPI	Khung web dựng các endpoint của issuer, ví và verifier.	https://fastapi.tiangolo.com/
Uvicorn	Máy chủ chạy ứng dụng FastAPI.	https://www.uvicorn.org/
Pydantic	Kiểm tra và ràng buộc kiểu dữ liệu vào ra của API.	https://pypi.org/project/pydantic/
py_webauthn	Xác minh passkey theo chuẩn WebAuthn.	https://pypi.org/project/webauthn/
google-auth	Kiểm ID-token khi đăng nhập bằng tài khoản Google.	https://pypi.org/project/google-auth/
web3.py	Đọc khoá công khai và schema từ hợp đồng trên chain.	https://pypi.org/project/web3/
websockets	Kênh thời gian thực giữa máy tính và điện thoại khi quét giấy tờ.	https://pypi.org/project/websockets/

A.2 Phía trình duyệt — TypeScript

Thư viện	Vai trò	Địa chỉ
React	Dựng giao diện ví và cổng verifier.	https://react.dev/
Vite	Công cụ đóng gói và máy chủ phát triển.	https://vite.dev/
TypeScript	Ngôn ngữ có kiểu tĩnh, bắt lỗi trước khi chạy.	https://www.typescriptlang.org/
Tailwind CSS	Hệ thống tạo kiểu giao diện.	https://tailwindcss.com/
Framer Motion	Hiệu ứng chuyển cảnh giữa các bước.	https://motion.dev/
jsQR	Đọc mã QR in trên mặt trước giấy tờ.	https://www.npmjs.com/package/jsqr

Thư viện	Vai trò	Địa chỉ
Tesseract.js	Đọc chữ trên ảnh giấy tờ, bù các trường mã QR không chứa.	https://tesseract.projectnaptha.com/
qrcode	Sinh mã QR chuyển phiên từ máy tính sang điện thoại.	https://www.npmjs.com/package/qrcode
React Router	Điều hướng giữa các màn hình.	https://reactrouter.com/

Phụ lục B

Source code

Mã nguồn đầy đủ của ứng dụng được lưu tại kho Git của dự án. Bảng dưới đây đối chiếu từng phần lý thuyết trong tài liệu với vị trí hiện thực tương ứng.

Nội dung	Vị trí trong mã nguồn
Sinh khoá, schema, ký mù	<code>issuer/issuer.py</code>
Link secret, cam kết, sigma protocol	<code>wallet/wallet.py</code>
Ngẫu nhiên hoá và tạo bằng chứng	<code>wallet/wallet.py</code>
Phép kiểm bằng chứng	<code>verifier/verifier.py</code>
Mã hoá thuộc tính, hàm băm thách thức	<code>common.py</code>
Đọc ghi hợp đồng trên chain	<code>chain.py</code>
Xác minh passkey (WebAuthn)	<code>passkey.py</code>
Giao diện ví và cổng verifier	<code>frontend/src/apps/</code>

Phụ lục C

Video demo và hướng dẫn sử dụng

Phần này sẽ được bổ sung sau.